ARAŞTIRMA SORULARI

1.Git nedir? GitHub nedir?

Git,projenizdeki dosya değişikliklerini anlık olarak kaydeden bir versiyon kontrol sistemidir.Bilgisayarınızda yerel olarak çalışır.
GitHub ise Git ile takip ettiğiniz bu dosyaların internet üzerinde(bulutta) depolanmasını,paylaşılmasını ve diğer geliştiricilerle iş birliği yapılmasını sağlayan bir platformdur.

Git&GitHub ne işe yarar?
Kod yazarken bazen yaptığınız değişikler sistemi bozabilir.Git sayesinde projenizin çalışan eski bir haline saniyeler içinde geri dönebilirsiniz.GitHub ise projelerinizi yedeklememize,açık kaynak dünyasında katkıda bulunmanıza ve profesyonel portfolyonuzu sergilememize olanak tanır.





2.Repository nedir?

Bilgi,belge,kod veya verilerin güvenli bir şekilde saklandığı,düzenlendiği ve gerektiğinde erişebildiği depo,arşiv veya havuz anlamına gelir.

3.Working directory ve staging area ifadelerini araştırın.

Working Directory(çalışma dizini)
–Projenizin bilgisayarınızdaki fiziksel klasörüdür.Kod editörünüzü (VS Code vb.)açıp bir dosyada değişiklik yaptığınızda,bu değişiklik doğrudan Working Directory içinde gerçekleşir.

-Henüz Git'in kalıcı hafızasına alınmamış,üzerinde çalışılmakta olan"ham" aşamadır.

-Git status komutunu yazdığınızda,burada yapılan değişiklikler genellikle kırmızı renkte görünür.

Staging Area(Hazırlık alanı veya ındex)
–Git'in en benzersiz özelliklerinden biridir.Working Directory ile local repository arasında bir köprü veya bekleme odası görevi görür.

-Değişiklik yaptığınız onlarca dosyadan sadece belirlediklerinizi bir sonraki kayda(commit)eklemek için buraya taşırsınız.

-Bir dosyayı git add <dosya_adi> komutuyla buraya gönderdiğinizde,dosya Staging Area'ya geçer.

-Git status komutunu yazdığınızda,hazır bekleyen bu dosyalar genellikle yeşil renkte görünür.

Kavram	Nedir?	Git Dilindeki Karşılığı	Ne Zaman Kullanılır?
Working Directory (Çalışma Dizini)	Bilgisayarınızda doğrudan **gördüğünüz ve düzenlediğiniz** klasördür.	**Tracked**(takip edilen veya **Untracked** (Takip edilemeyen) dosyalar.	Kod yazarken, dosya eklerken veya silerken.
Staging Area/ Index (Hazırlık Alanı)	Git'e kaydetmek (commit) istediğiniz değişikliklerin **geçici olarak toplandığı** yerdir.	**Staged** (hazırlanmış) dosyalar.	Bir sonraki kayıt paketine dahil edilecek dosyaları seçerken.
Local Repository (Yerel Depo)	Değişikliklerin kalıcı olarak **tarihçeye kaydedildiği**(.git klasörü)yerdir.	**Committed** (kaydedilmemiş dosyalar.)	Yapılan işi kalıcı bir versiyon (snapshot) haline getirirken.

4.Commit neyi temsil eder?

Commit:
Git'e eklediğimiz dosyaları kalıcı olarak Git veri tabanına işlemeye commit denir.Projede belli başlı işlemler yapıldıktan sonra dosyaları commit etmek gerekir.Örnek olarak üye kayıt-giriş işlemlerini hallettikten sonra bir commit alabilirsiniz.

Commit neyi temsil eder?
-Bir anlık görüntüyü(Snapshot):
–Projenin o andaki durumunu,dosyaların tam halini ve yapılan ekleme/çıkarma/güncellemeleri paketler.

-Zaman çizelgesinde bir durağı:
–Projenin geçmişindeki belirli bir noktayı işaret eder.Gerekirse kodun bu geçmiş sürümlerine kolayca geri dönülmesini sağlar.

-Net bir açıklamayı(commit message):
–O değişikliklerin neden ve ne amaçla yapıldığını anlatan bir not içerir.

-Ekip içi sorumluluğu:
–Kimin hangi değişikliği yaptığını ve projenin hangi aşamadan geçtiğini takip etmeyi sağlar.

5.Push ve pull ne yapar?

-Git Push:
–Kendi bilgisayarınızda (yerel deponuzda) yaptığınız ve kaydettiğiniz(commit) kod değişikliklerini,GitHub gibi bulut veya uzak sunucudaki(remote) depoya gönderir.

-Git Pull:
–Uzak sunucudaki depoda yer alan güncellemeleri,kodları veya diğer geliştiricilerin yaptığı değişiklikleri bilgisayarınıza indirir ve yerel projenizle otomatik olarak birleştirir (merge).

6.Branch neden vardır? Merge ne zaman gerekir?

Yazılım geliştirme süreçlerinde branch(dal),ana koddan(örneğin main veya master)bağımsız bir çalışma alanı oluşturmak oluşturmak için vardır.

-İzolasyon sağlar:
–Yeni bir özellik(feature),hata düzeltmesi(bugfix) veya deneyimsel bir çalışma,ana kodu bozmadan ayrı bir dalda geliştirebilir.

-Çakışmaları önler:
–Birden fazla geliştirici,aynı dosyalarda birbirini etkilemeden aynı anda çalışabilir.

-Kopyalamayı önler:
–Kodun kopyalarını alıp “proje_son_kopya_v2.zip” gibi karmaşalar yaratmak yerine temiz bir geçmiş sunar.

Marge ne zaman gerekir?

-Özellik tamamlalandığında:
–Yeni bir özellik branch üzerinde kodlanıp test edilip onaylandığında ana dala eklenmesi gerekir.

-Hata düzeltildikten sonra:
–Canlıdaki bir hatayı düzeltmek için açılan acil bir hotfix branch’indeki çözümün ana koda yansıtılması gerekir.

-Versiyon birleştirmede:
–Farklı ekiplerin yazdığı kodların ortak bir projede buluşması istendiğinde.

7...gitgnore dosyası hangi amaçla kullanılır?

-.gitgnore dosyası:
–Bir Git deposunda hangi dosya ve klasörlerin Git tarafından takip edilmeyeceğini(görmezden gelineceğini) belirleyen düz metin bir kural dosyasıdır.

Temel amaçları ve kullanım alanları

-Hassas bilgileri korumak:
–API anahtarları,şifreler,veritabanı bağlantı bilgileri veya gizli ayarlar içeren .env gibi dosyaların GitHub veya başka bir uzak depoya(remote repository)yanlışıkla yüklenmesini önler.

-Gereksiz dosya kalabalığnı önlemek:
–Paket yöneticileri tarafından indirilen bağımlılıklar(örneğin Node.js projelerindeki node_modules/klasörü)gibi başka bir bilgisayarda kolayca tekrar kurulabilen büyük klasörleri dışarıda bırakır.

-Depo boyutunu küçük tutmak:
–Video,büyük görsel dosyaları,log dosyaları,derleme çıktıları(dist/, build/) veya işletim sisteminin ürettiği geçici sistem dosyalarını(.DS_Store vb.)takip dışı bırakarak deponun gereksiz yere şişmesini engeller.

-Kişisel ayarları ayırmak:
–Kullanılan IDE’ye(.vscode/ vb.) veya kişisel çalışma ortamına özel ayar dosyalarının ekip üyeleri arasında gereksiz çakışmalara(marge conflict) yol açmasını önler.

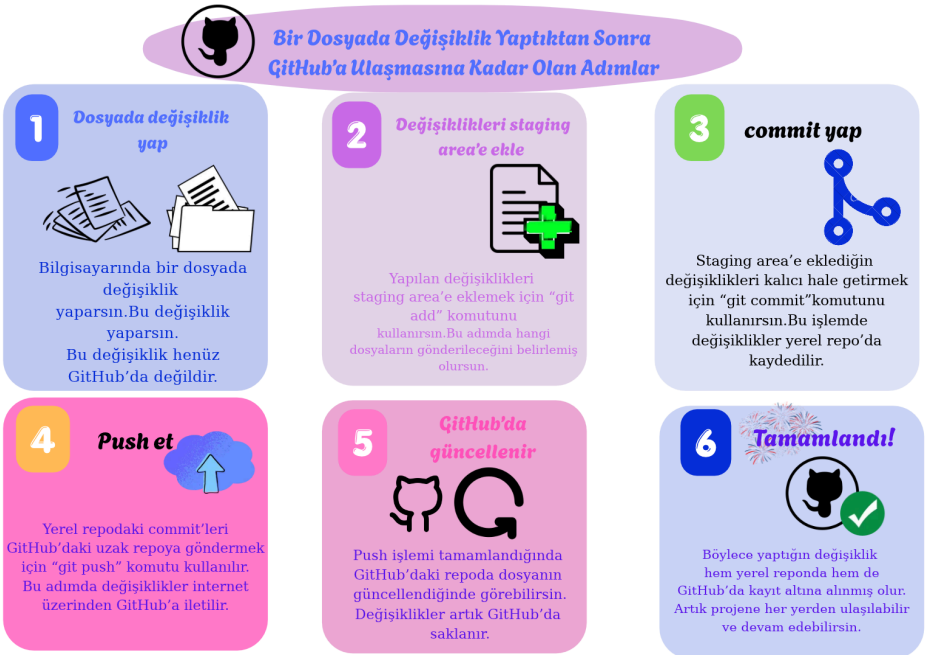
8.Neden her gün ayrı repository açmak yerine aynı repository üzerinde ilerlemek daha mantıklı olabilir?

Her gün ayrı repository açmak yerine aynı repository üzerinde ilerlemek daha mantıklıdır.Çünkü bütün çalışmalar tek bir yerde düzenli şekilde tutulur.Önceki günlerde yapılan araştırmalara kolayca ulaşılabilir ve çalışmalar arasındaki bağlantı korunur.Ayrıca her gün yapılan değişiklikler commit edilerek takip edilebilir.Böylece GitHub üzerinde daha düzenli bir çalışma ortamı oluşur ve dosyaların karışması önlenir.

ÖRNEK:
Her gün için ayrı bir repository açmak yerine aynı repository içinde 06-10-csharp.md,12-10-web.md ve 13-10-veritabani.md gibi farklı dosyalar oluşturulabilir.Böylece bütün staj notları tek bi yerde toplanır.

Gün sonunda teslim edeceğiniz not:

1.Bir dosyada değişiklik yaptıktan sonra bunun GitHub'a ulaşmasına kadar olan adımları çizin:



2.Commit ile push arasındaki farkı kendi cümlenizle yazın.

Commit,yaptığım değişiklikleri bilgisayarımdaki yerel git deposuna kaydetmemi sağlar.Push ise commit ile kaydettiğim değişiklikleri GitHub'daki uzak depoya

göndermemi sağlar.

ÖRNEK:
Commit değişiklikleri bilgisayarıma kaydetmek,push ise bu değişiklikleri GitHub'a göndermek gibidir.